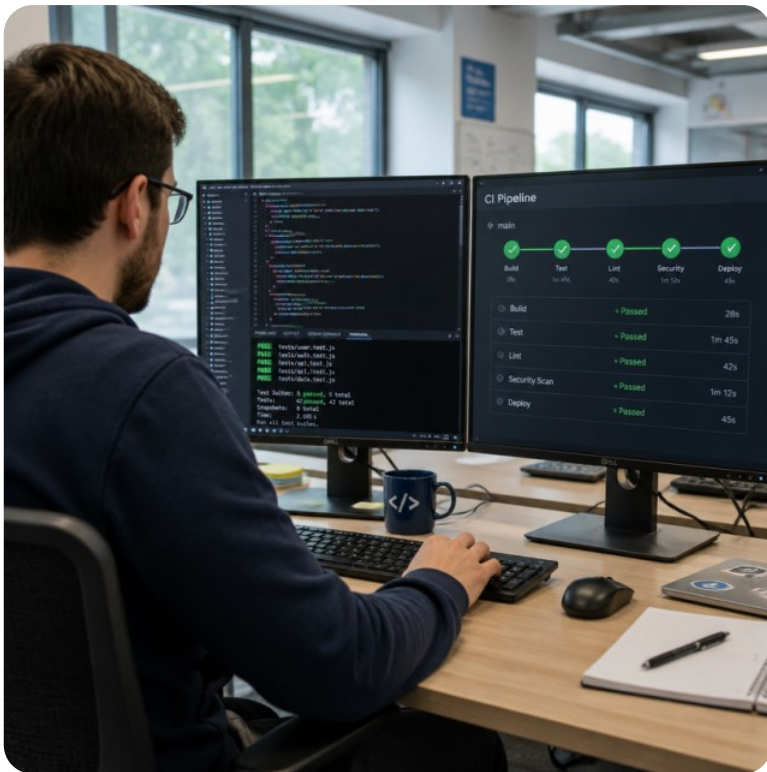




Projeto Prático Integrador: Suíte de Testes

Construindo uma suíte completa de qualidade para front-end.

O Desafio da Produção

Sua aplicação aguenta o fluxo real de compras na Black Friday sem quebrar nenhuma funcionalidade crítica?



Objetivos da Aula

METAS DE APRENDIZAGEM

Ao término, você será capaz de:

- Estruturar a arquitetura de uma suíte de testes.
- Implementar testes unitários, integração e E2E.
- Automatizar via CI/CD com 80% de cobertura.



Ponto-chave

Qualidade depende da integração da pirâmide de testes.



Vocabulário Essencial de QA



Arquitetura

Organização estrutural dos arquivos, bibliotecas e camadas da suíte.



Integração

Validação da comunicação coordenada entre múltiplos componentes e serviços.



Escopo

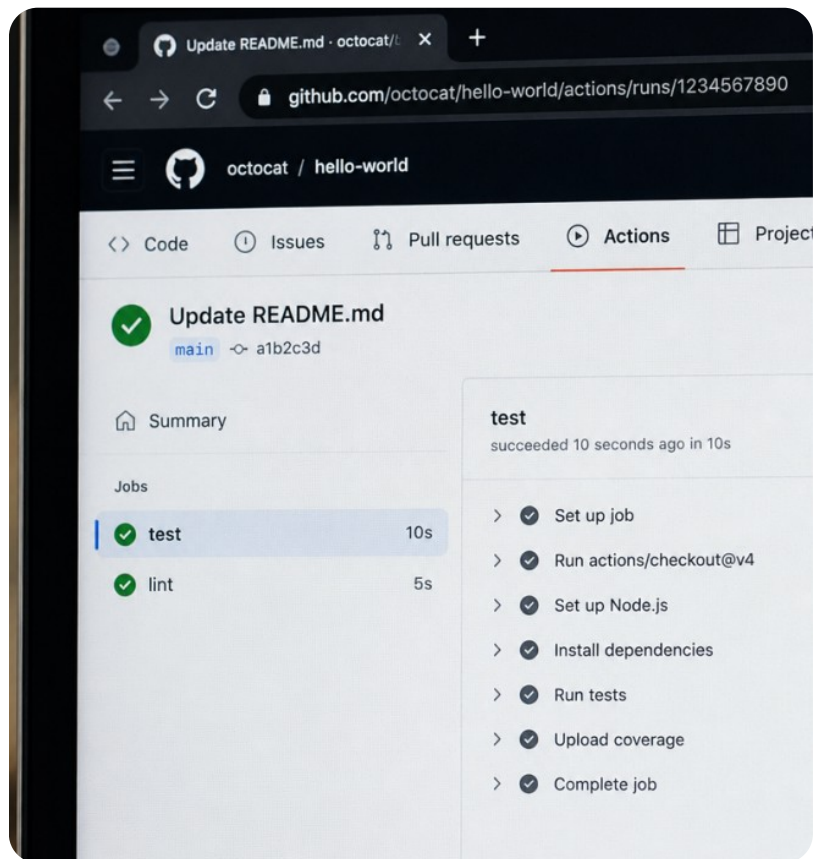
Fronteira bem definida das funcionalidades e fluxos cobertos pela automação.



Avaliação

Medição técnica de conformidade, cobertura de código e eficácia da suíte.

Recapitulação: CI/CD no GitHub



REVISÃO DA AULA 9

Vimos que a **Integração Contínua (CI)** evita regressões.

GitHub Actions dispara jobs a cada PR.
Falhas em testes **bloqueiam o merge**.

- Garante estabilidade em produção.



Lembre-se

O pipeline de CI é o guardião da qualidade do software.

Acessibilidade e Performance



Padrões WCAG 2.1

Testes automatizados com ferramentas como axe-core validam contrastes de cores, roles semânticas ARIA e acessibilidade via teclado para todos os usuários.



Core Web Vitals

Métricas como LCP e CLS garantem tempos de resposta rápidos e estabilidade visual na interface, reduzindo atritos durante a conversão do usuário.

Quiz Rápido de Fixação



Pergunta 1:

Qual é a função primordial de um workflow de CI no GitHub Actions em Pull Requests?

Pergunta 2:

O que avaliam as diretrizes internacionais WCAG 2.1 no desenvolvimento front-end?

Pergunta 3:

Por que relatórios automatizados de performance como Core Web Vitals importam no pipeline?

Quiz Rápido de Fixação



Resposta 1:

Executar testes e validações automaticamente para bloquear merges que introduzam quebras.

Resposta 2:

A conformidade de acessibilidade para garantir que pessoas com deficiência naveguem com autonomia.

Resposta 3:

Porque detectam lentidões no carregamento e instabilidades de layout antes do lançamento em produção.

Apresentação do Projeto Capstone

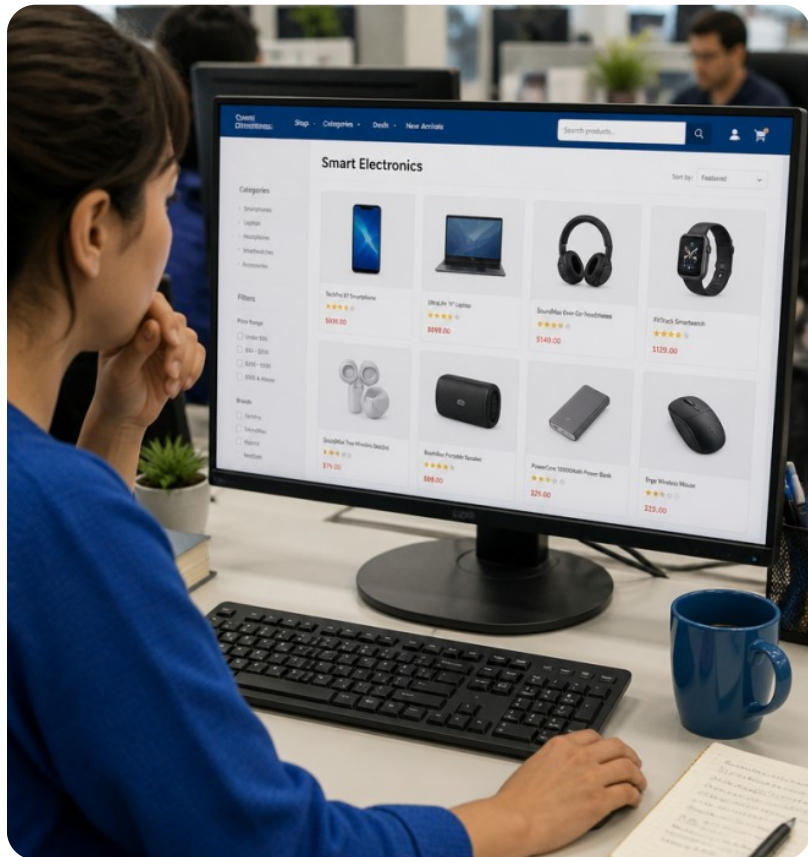
Nosso projeto capstone consiste em construir e blindar a suíte completa de QA para uma **aplicação E-commerce** realista.

A plataforma inclui catálogo de produtos, cálculo dinâmico de carrinho, cupons promocionais e fluxo completo de checkout.



Ponto-chave

O objetivo é aplicar todas as ferramentas estudadas no curso em um único repositório profissional.



Escopo e Requisitos do Projeto

METAS DE QUALIDADE

O projeto exige:

Cobertura $\geq 80\%$ (linhas/branches/funções).

- Testes unitários p/ regras de preço.
- Testes de integração p/ carrinho.

Fluxo **E2E crítico** automatizado.



Atenção

Cobertura de 80% é requisito obrigatório no pipeline.

```
developer@istanbul-project:~/project$ npx istanbul report
Coverage summary:
+-----+-----+-----+-----+-----+
| File           | % Branch | % Statement | % Line | % Functions |
+-----+-----+-----+-----+-----+
| All files      | 86.21    | 88.42       | 89.11  | 90.00       |
+-----+-----+-----+-----+-----+
| src            | 86.21    | 88.42       | 89.11  | 90.00       |
| src/api        | 85.00    | 87.50       | 88.10  | 89.47       |
| src/api/user.js | 90.91    | 91.30       | 91.67  | 92.86       |
| src/api/auth.js | 84.62    | 86.36       | 87.10  | 88.89       |
| src/utils      | 87.50    | 89.47       | 90.48  | 90.91       |
| src/utils/helpers.js | 88.89    | 90.91       | 91.67  | 91.67       |
| src/utils/validator.js | 86.36    | 88.00       | 89.29  | 90.00       |
+-----+-----+-----+-----+-----+

developer@istanbul-project:~/project$
```

Pirâmide de Testes no E-commerce



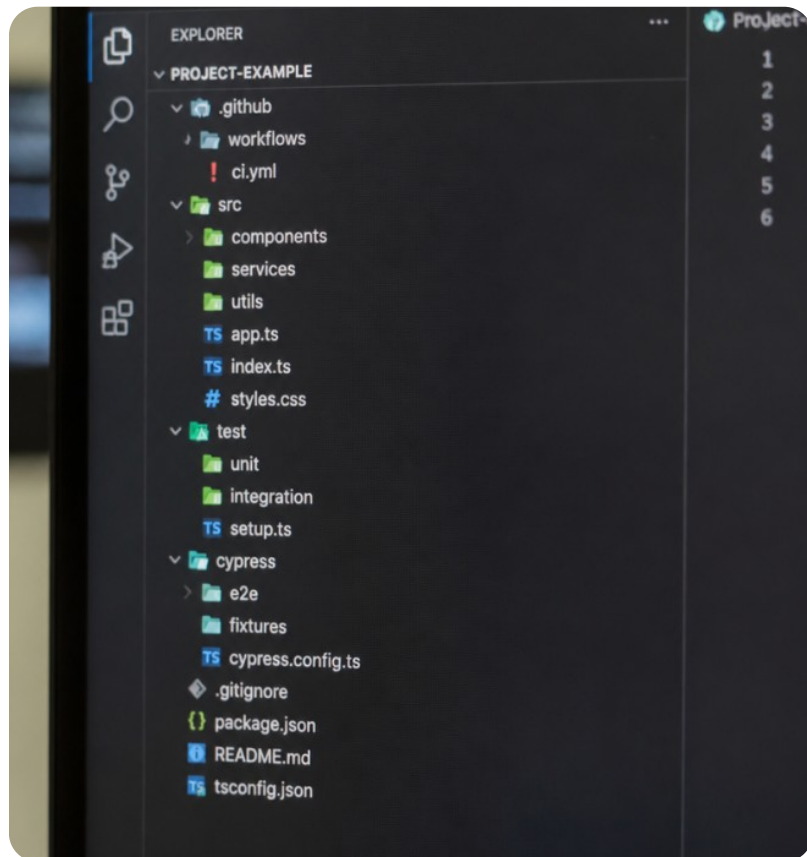
Exemplo

No nosso e-commerce: 25 testes unitários de cálculo, 10 testes de integração do carrinho e 2 jornadas E2E completas de checkout.

Estrutura Proporcional

A base unitária com Vitest garante execução em milissegundos. A camada média integra componentes e DOM, enquanto o topo valida a experiência final do cliente no navegador.

Arquitetura de Arquivos



ESTRUTURA DE PASTAS

Separar produção de testes facilita a manutenção:

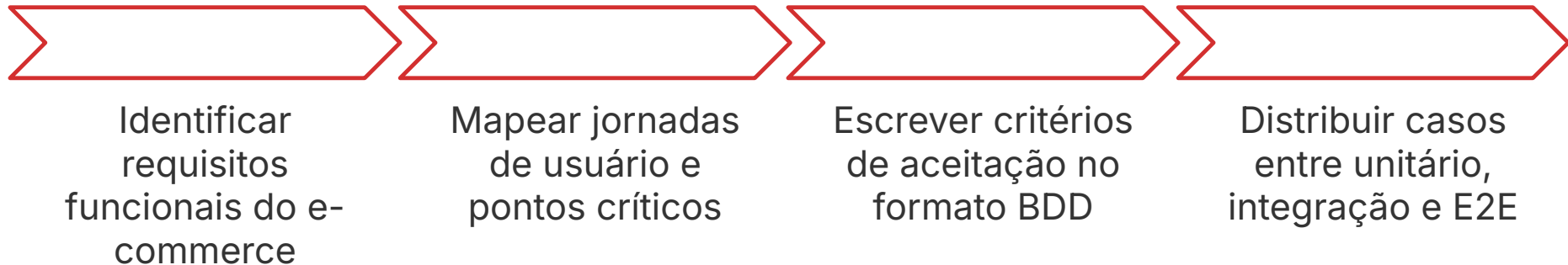
- : CI/CD com Actions.
- : código-fonte (componentes, hooks, utils).
- : unitários e integração (Vitest, MSW).
- : fluxos E2E e Page Objects.



Lembre-se

Arquitetura previsível agiliza onboarding e automação.

Mapeando o Plano de Testes



Jornada Crítica: O Funil de Compra

FLUXO CRÍTICO

O funil de maior impacto financeiro:

- Busca e seleção de produto.
- Aplicação de cupom e recálculo.
- Endereço e cálculo de frete.
- Pagamento e comprovante.



Atenção

Falhas aqui causam abandono de carrinho e prejuízo direto.



Testes Unitários com Vitest

CAMADA UNITÁRIA

As regras de negócio puras devem ser isoladas de UI e banco de dados:

Funções como `add` e `multiply`.

O padrão **AAA (Arrange, Act, Assert)** organiza cada cenário.

Usamos `describe` para agrupar e `it` para asserções precisas.

```
JS math.test.js x
tests > JS math.test.js > ...
1 import { describe, it, expect } from 'vitest'
2 import { add } from '../src/math.js'
3
4 describe('add', () => {
5   it('should return the sum of two numbers', () => {
6     // Arrange
7     const a = 2
8     const b = 3
9
10    // Act
11    const result = add(a, b)
12
13    // Assert
14    expect(result).toBe(5)
15  })
16 })
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS TEST RESULTS

Test Files 1 passed (1)
Tests 1 passed (1)
Start at 10:42:18
Duration 28ms



Exemplo

Validamos que um cupom de 10% nunca resulte em valor negativo, mesmo com descontos combinados.

Regras de Preço e Descontos

Casos Típicos

- Produtos com quantidades normais somam corretamente.
- Cupons válidos reduzem o subtotal conforme a porcentagem estipulada.
- O frete fixo é adicionado sem incidir sobre o desconto promocional.

Casos de Borda (Edge Cases)

- Carrinho vazio com cupom não deve quebrar a aplicação.
- Cupom expirado deve lançar erro legível ou ser rejeitado.
- Arredondamento monetário deve preservar duas casas decimais com precisão.



Ponto-chave

Edge cases são os responsáveis pela maior parte dos bugs invisíveis em produção.

Validação de Teste Unitário

Testes unitários de regras de preço devem renderizar o componente React do carrinho para calcular o total.



VERDADE
IRO



FALSO



Prepare-se para explicar o seu raciocínio.

Validação de Teste Unitário



Testes unitários de regras de preço devem renderizar o componente React do carrinho para calcular o total.



Por que é isso?

Regras de cálculo são funções puras em JavaScript e devem ser testadas sem dependência do DOM ou componentes visuais.

Mocking com Mock Service Worker

ISOLAMENTO DE REDE

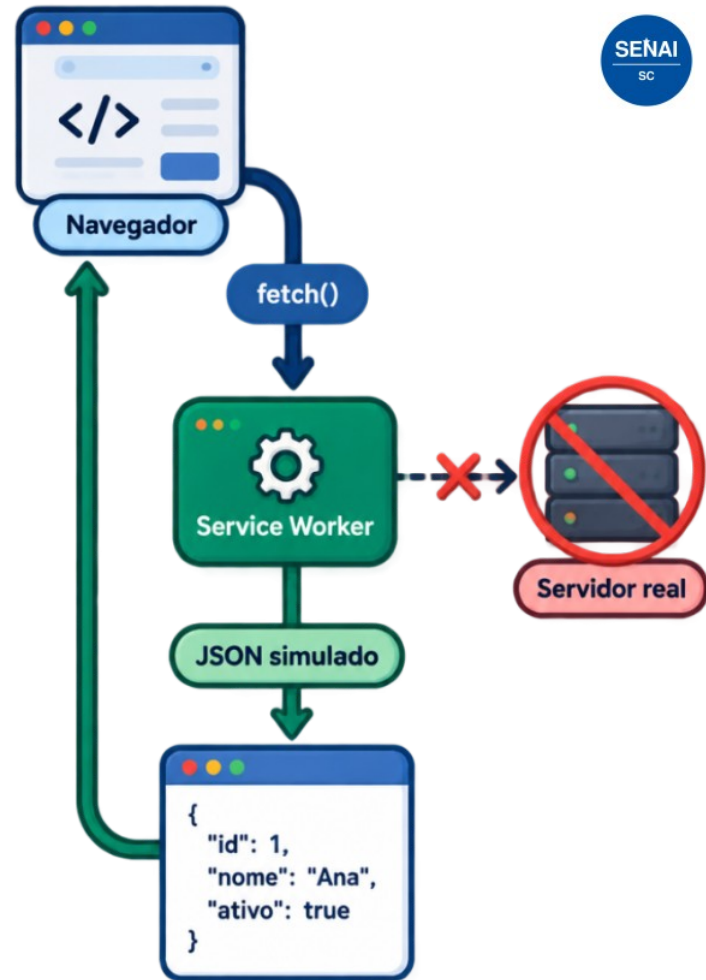
Simular APIs com **MSW** traz benefícios:

- Intercepta requisições HTTP na camada de rede.
- Não altera código de produção com mocks.
- Simula latência, erros 500 e payloads dinâmicos.

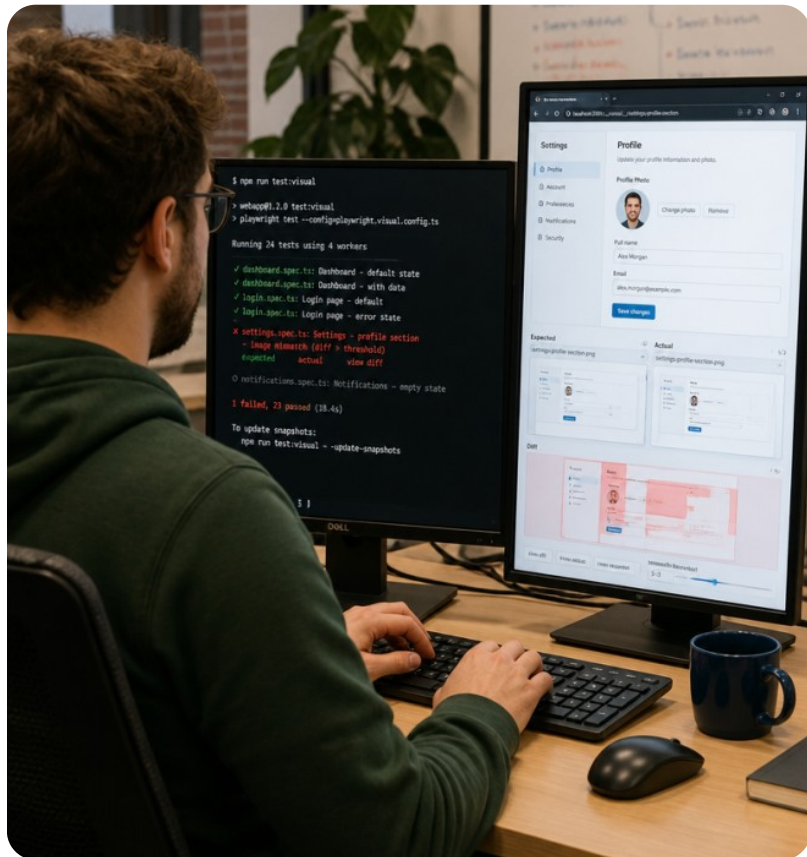


Lembre-se

MSW garante testes determinísticos imunes a falhas no backend.



Integração com Testing Library



TESTE DE COMPORTAMENTO

A **Testing Library** prioriza o usuário:
Usa e .

- Evita seletores baseados em CSS.

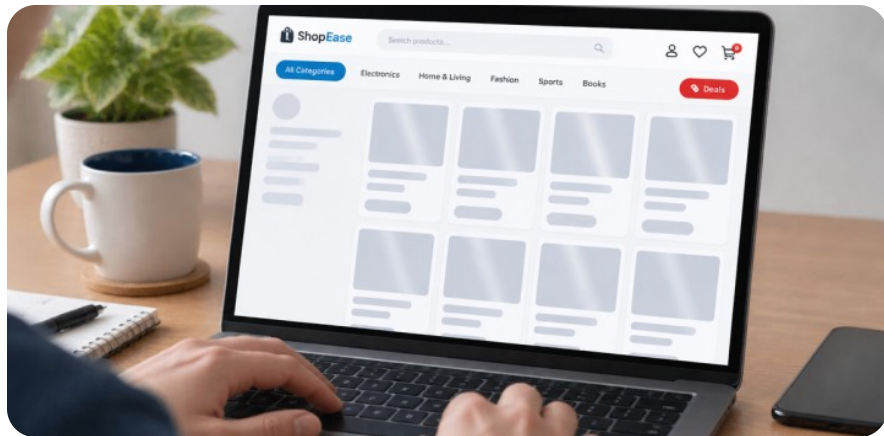
Simula eventos reais com .
Valida o via cliques.



Ponto-chave

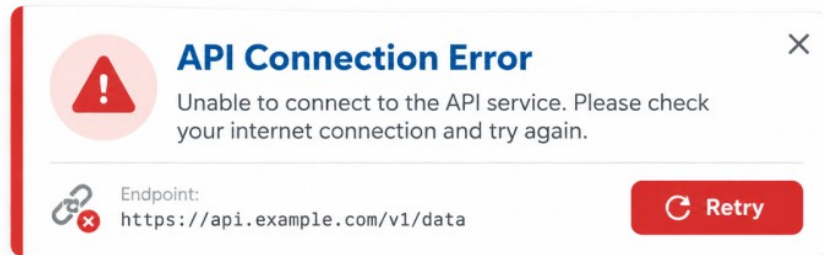
Teste interações, não a implementação.

Simulando Estados Assíncronos



Estado de Carregamento

Usamos com asserções assíncronas para checar spinners de carregamento e desabilitação preventiva de botões durante requisições.



Tratamento de Erro

Configuramos handlers de erro no MSW para validar se o componente exibe mensagens claras de falha sem interromper a interface.

Prática com Testing Library

Para consultar elementos com acessibilidade usamos ____, e para simular digitação realista do usuário utilizamos ____.

Banco de palavras 

userEvent

getElementById

fireClick

getByRole

querySelector

Prática com Testing Library



Para consultar elementos com acessibilidade usamos **getByRole**, e para simular digitação realista do usuário utilizamos **userEvent**.

Banco de palavras 🏛️

userEvent

getElementById

fireClick

getByRole

querySelector

Panorama dos Testes E2E



Automação E2E: Cypress e Playwright

CAMADA END-TO-END

Cypress e Playwright são líderes em E2E:

- Executam em navegadores reais com vídeo e screenshots.

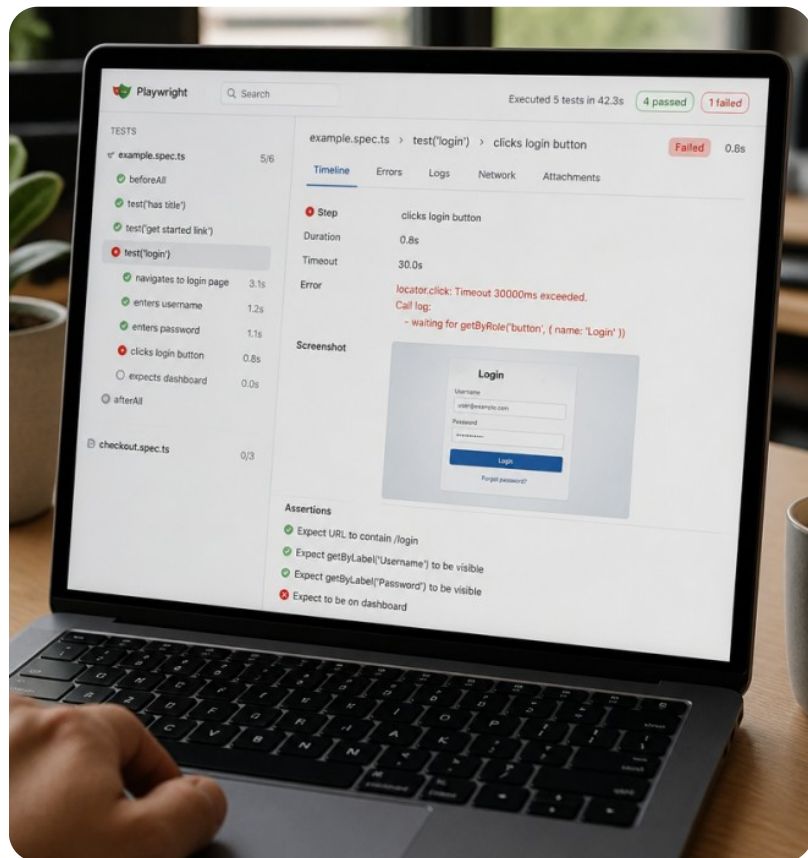
Oferecem **time-travel debugging** para inspecionar o DOM.

- Aguardam elementos automaticamente sem sleeps manuais.

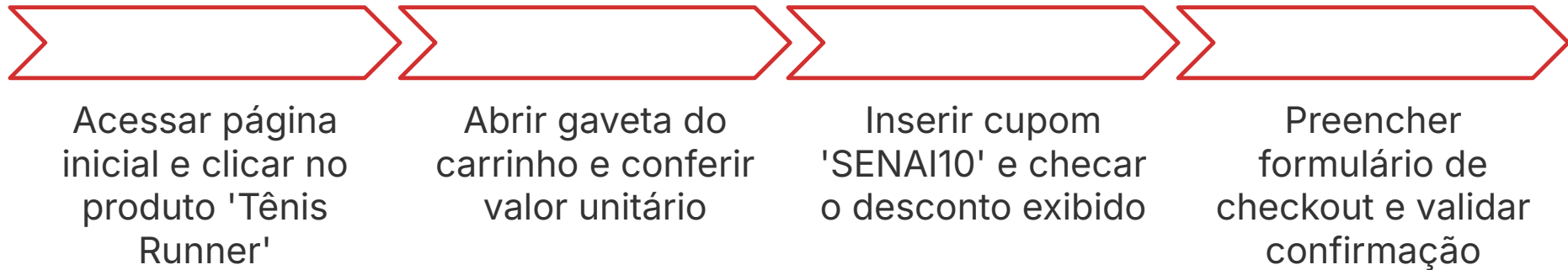


Exemplo

Um teste E2E valida desde a adição do item até a confirmação bancária.



Fluxo Completo de Checkout



Ordem de Execução do E2E

Ordene os passos da automação do fluxo de compra no teste E2E:

Finalizar compra e validar a mensagem de sucesso

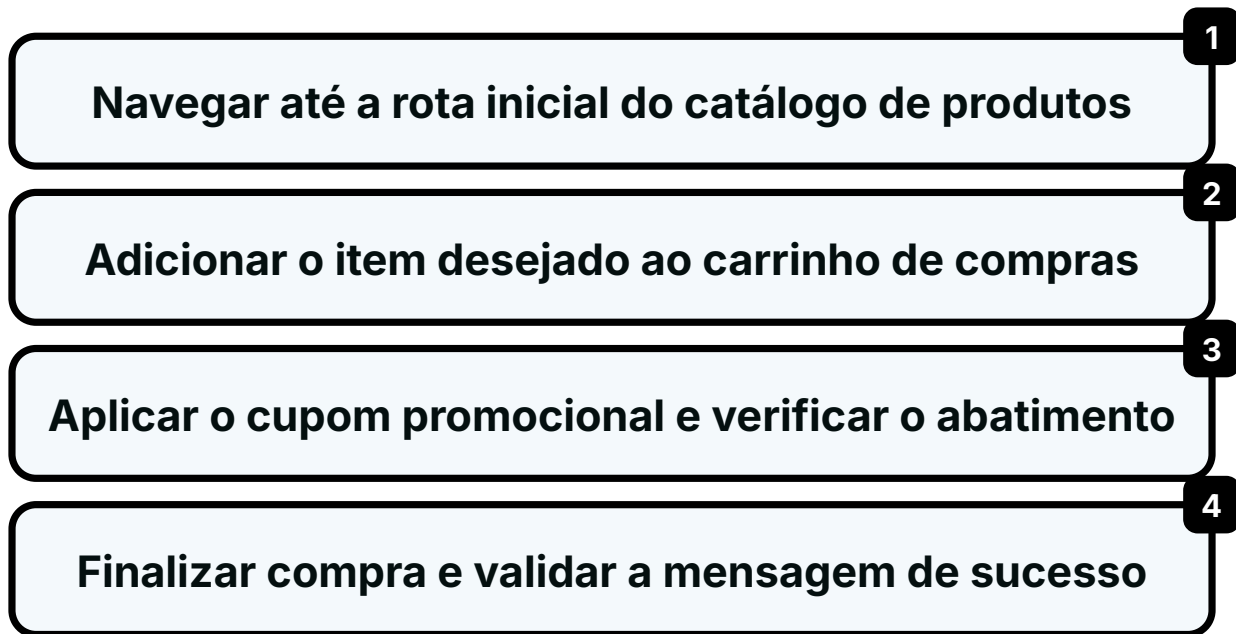
Navegar até a rota inicial do catálogo de produtos

Aplicar o cupom promocional e verificar o abatimento

Adicionar o item desejado ao carrinho de compras

Ordem de Execução do E2E

Ordene os passos da automação do fluxo de compra no teste E2E:



Automação com GitHub Actions

INTEGRAÇÃO CONTÍNUA

O orquestra os testes:

Disparado em e .

- Cria ambiente Node.js e instala dependências. Executa e testes E2E.
- Envia relatórios de cobertura.



Lembre-se

Falha em testes bloqueia o merge do PR e protege a branch main.



Análise da Cobertura de Código

MÉTRICAS DE QUALIDADE

Alcançar 80% exige estratégia:

Line Coverage: linhas executadas nos testes.

Branch Coverage: caminhos testados.

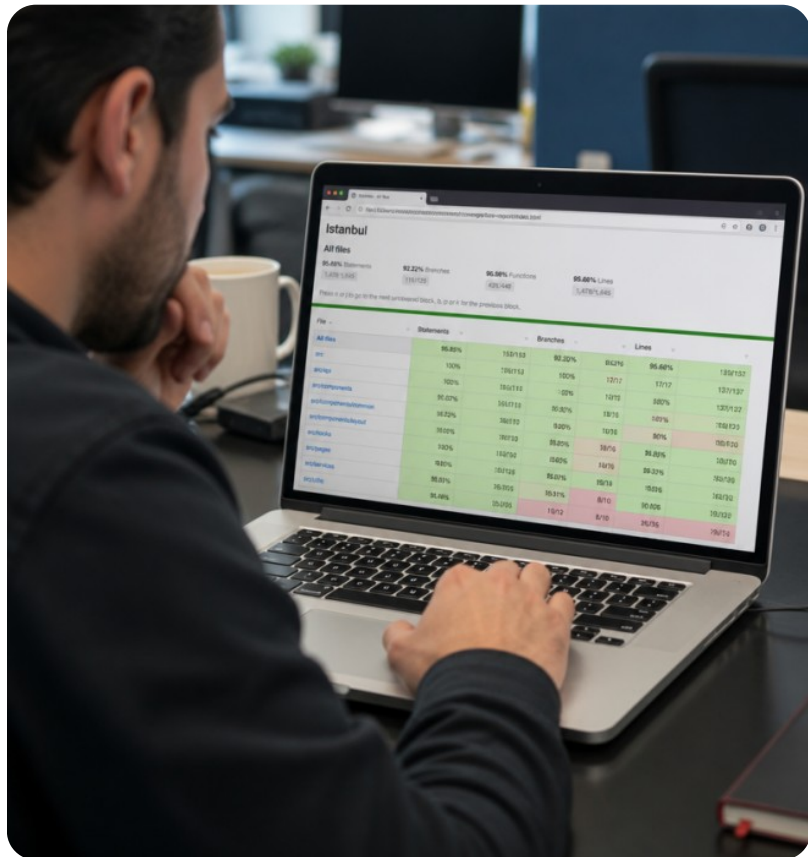
Function Coverage: funções acionadas.

- Gaps no relatório Istanbul guiam novos testes.



Atenção

100% de cobertura não garante ausência de bugs; 80% com bons cenários é o ideal.



Alcançando a Meta de 80%



Gaps Comuns (<80%)

Falta de testes para tratamentos de erro em chamadas assíncronas, blocos ignorados e ramificações de validação de formulários não cobertas.



Estratégia de Ajuste

Adicionar testes unitários focados nas funções utilitárias e simular respostas HTTP 400/500 no MSW para cobrir os fluxos de exceção.

Trabalho em Duplas: Mão na Massa

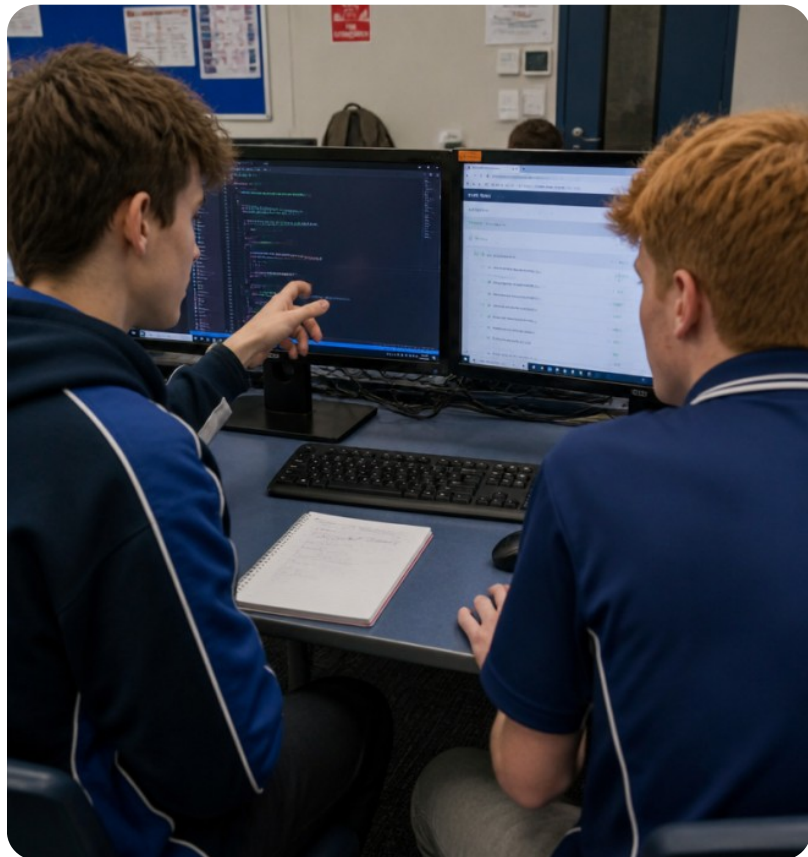
Agora é o momento de consolidar o projeto em duplas no laboratório.

Cada dupla clonará o repositório base do e-commerce, estruturará a suíte de testes com as 3 camadas e garantirá que o workflow do GitHub Actions termine totalmente verde.



Ponto-chave

Colaboração técnica, divisão clara de tarefas e foco no escopo definido são essenciais.



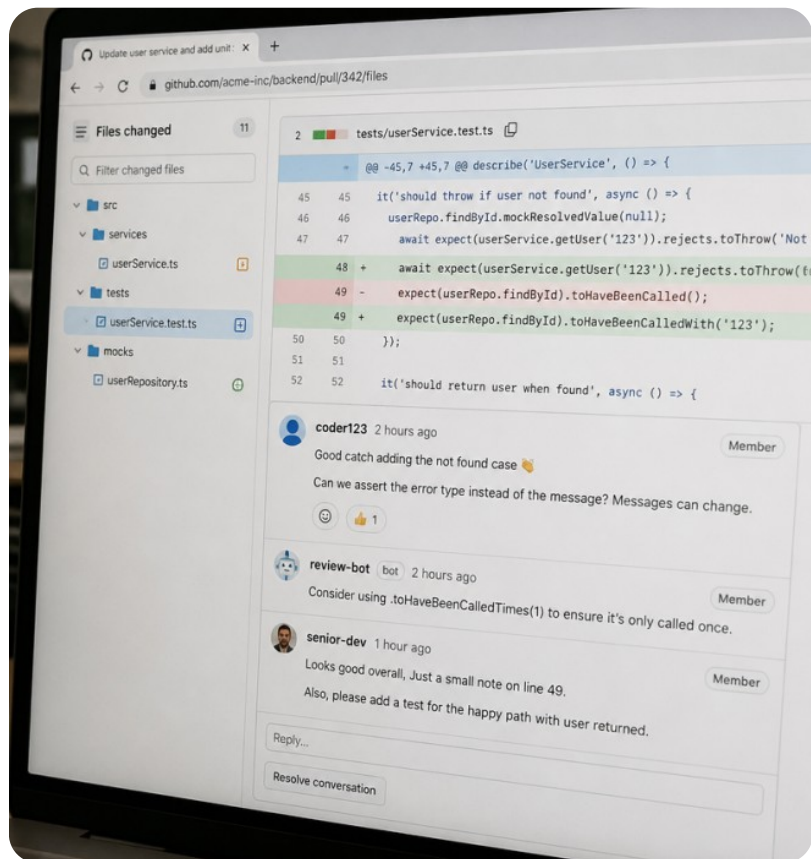
Atividade Prática: Suíte Integrada



Instruções para as duplas:

- 1.Clone 'senai-e-commerce-qa'.
- 1.Crie testes unitários para 'calculateDiscounts' (0%, 10%, 50%).
- 1.Teste integração do carrinho com MSW.
- 1.Rode 'npm run test:coverage' (cobertura $\geq$ 80%).
- 1.Abra PR com link do workflow verde do GitHub Actions.

Code Review da Suíte



REVISÃO POR PARES

Review de testes exige rigor igual ao código de produto:

- Teste é legível e expressa a regra?
- Há dependências ou ordem fixa?
- Evitamos testes intermitentes (flaky)?
- Asserções focam em comportamento, não implementação?



Lembre-se

Testes mal escritos elevam custos e geram falsa segurança.

Discussão: Flakiness e Manutenção



Um teste E2E do carrinho passa 9 vezes e falha 1 vez sem alteração de código. O que a equipe deve fazer?

Discussão: Flakiness e Manutenção



Você poderia ter dito...

Ignorar testes intermitentes destrói a confiança no CI.

Isole e investigue falhas imediatamente.

Causas: esperas fixas em vez de seletores DOM,
concorrência e dados voláteis no banco.

Avaliação Geral do Curso

SÍNTESE DA JORNADA

Em 10 aulas, consolidamos bases:

- Pirâmide, TDD e ferramentas líderes.
- Vitest, Testing Library, MSW, Cypress e Playwright.
- Auditorias Lighthouse e CI/CD com GitHub Actions.
- Foco em confiabilidade contínua.



Ponto-chave

Testar garante qualidade desde a concepção.

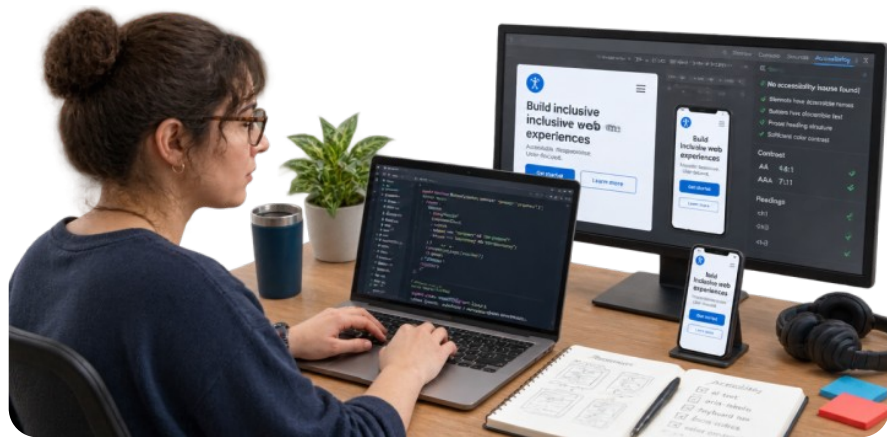


Carreira em Garantia de Software



QA Engineer / SDET

Foco em automação de ponta a ponta, infraestrutura de CI/CD, testes de carga e criação de frameworks de teste para múltiplos times de engenharia.



Frontend Engineer

Desenvolvedores com cultura de testes e TDD são altamente disputados no mercado por entregarem código limpo, estável e de fácil manutenção.

Verificação Final de Aprendizado

Qual é a combinação recomendada para garantir alta confiança e rapidez na suíte de testes de um front-end moderno?

1. Testes unitários cobrindo o DOM sem validação de regras de negócio ou de integração.
2. Apenas testes E2E no Cypress, dispensando testes unitários por serem muito simples.
3. 100% de testes manuais em ambiente de staging antes de cada lançamento.
4. Muitos testes unitários rápidos, boa camada de integração com Testing Library e poucos fluxos críticos E2E automatizados.

Verificação Final de Aprendizado



Qual é a combinação recomendada para garantir alta confiança e rapidez na suíte de testes de um front-end moderno?

1. Testes unitários cobrindo o DOM sem validação de regras de negócio ou de integração.
2. Apenas testes E2E no Cypress, dispensando testes unitários por serem muito simples.
3. 100% de testes manuais em ambiente de staging antes de cada lançamento.
4. Muitos testes unitários rápidos, boa camada de integração com Testing Library e poucos fluxos críticos E2E automatizados.

Qualidade é Prática Contínua

Parabéns por concluir a disciplina de Testes de Front-end! Levem a cultura de automação, acessibilidade e rigor técnico para todos os projetos da sua carreira.

